TypeScript 类型体操基础

刘文涛 | 2023年10月20日创建

什么是类型体操

类型体操是一种调侃的说法，其来源于TypeScript中各种复杂的类型操作。

TypeScript 类型体操是指在使用 TypeScript 编写代码时，通过灵活运用类型系统来实现一些有趣的、复杂的类型操作，以确保代码的类型安全和正确性。

类型基础

基础类型

- 简单数据类型：
  - 数字类型：`let age: number = 30;`
  - 字符串类型：`let name: string = "John";`
  - 布尔类型：`let isStudent: boolean = true;`
  - 空值和未定义：`let something: null = null;`、`let somethingElse: undefined = undefined;`
- 类型推断和显式类型注解：
  - TypeScript 可以根据上下文自动推断变量类型，但也可以显式指定类型。
- 类型断言：
  - 使用类型断言可以告诉 TypeScript 某个值的类型。

复杂类型

- 对象类型：
  - 定义对象类型和属性：`let person: { name: string, age: number } = { name: "Alice", age: 25 };`
- 数组类型：
  - 数组类型注解：`let numbers: number[] = [1, 2, 3];`
- 函数类型：

- 定义函数类型和参数：`let add: (a: number, b: number) => number = (a, b) => a + b;`

- 接口与类型别名：
  - 创建接口和类型别名用于复杂类型定义。
- 联合类型(Union)与交叉类型：
  - 联合类型示例：`let value: string | number = "Hello";`
  - 交叉类型示例：`let merged: { a: number } & { b: string } = { a: 1, b: "text" };`

高级类型

就像 JavaScript 的高阶函数是生成函数的函数，React 的高阶组件是生成组件的组件一样，Typescript 的 高级类型 就是生成类型的类型。

TypeScript 高级类型是通过 type 定义的有类型参数（也叫泛型）的类型，它会对传入的类型参数做一系列的类型计算，产生新的类型。

```
type Pick<T, K extends keyof T> = {
    [P in K]: T[P];
};
```

比如，这个 Pick 就是一个高级类型，它有类型参数 T 和 K，类型参数经过一系列的类型计算逻辑，会返回新的类型。

泛型

泛型函数

泛型函数允许我们在函数定义中使用类型参数，以便在函数调用时动态指定类型。

```
function identity<T>(arg: T): T {
  return arg;
}

let result = identity<number>(42);  // result 的类型为 number
```

在上面的示例中，`identity` 函数使用类型参数 `T`，并返回与输入类型相同的值。通过显式传递泛型参数，我们可以确保在函数调用时指定了具体的类型。

```
function identity<T>(arg: T): T {
    return arg;
}
```

泛型接口

泛型接口允许我们在接口定义中使用类型参数，以便在实现该接口时指定具体的类型。

```
interface Container<T> {
  value: T;
}

let container: Container<number> = { value: 42 };
```

在上面的示例中，我们定义了一个泛型接口 `Container`，它包含一个类型参数 `T`。通过指定 `Container<number>`，我们创建了一个具体的实现，其中的 `value` 属性类型为 `number`。

泛型类

泛型类允许我们在类定义中使用类型参数，以便在创建类的实例时指定具体的类型。

```
class Stack<T> {
  private items: T[] = [];

  push(item: T) {
    this.items.push(item);
  }

  pop(): T | undefined {
    return this.items.pop();
  }
}

let stack = new Stack<number>();
stack.push(1);
stack.push(2);
let item = stack.pop();  // item 的类型为 number | undefined
```

在上面的示例中，我们定义了一个泛型类 `Stack`，它使用类型参数 `T` 来表示堆栈中的元素类型。通过创建 `Stack<number>` 的实例，我们限制了堆栈中的元素必须为 `number` 类型。

条件类型

条件类型允许我们根据输入类型的条件判断结果来选择不同的类型。条件类型的语法形式为：

```
T extends U ? X : Y
```

其中，`T` 是待检查的类型，`U` 是条件类型，`X` 是满足条件时返回的类型，`Y` 是不满足条件时返回的类型。

下面是一个使用条件类型的示例：

```
type Check<T> = T extends string ? true : false;
type Result = Check<string>;  // Result 的类型为 true
```

在上面的示例中，我们定义了一个条件

类型 `Check<T>`，它接受一个类型参数 `T`。如果 `T` 是 `string` 类型，那么 `Check<T>` 的类型将是 `true`，否则为 `false`。

映射类型

映射类型允许我们将一种类型映射为另一种类型. 常见的一种映射类型 `Readonly`

```
type Readonly<T> = { readonly [P in keyof T]: T[P]; }
```

递归类型

创建递归类型，如树结构。

```
type TreeNode<T> = {
  value: T;
  children?: TreeNode<T>[];
};
```

特性

类型断言

类型断言（Type Casting）是 TypeScript 中的一种特性，允许开发者手动指定变量的类型，即告诉编译器某个值的类型。它主要用于以下情况：

1. 处理类型不一致的情况： 当 TypeScript 的类型推断系统不能准确确定某个值的类型，但你明确知道该值的类型时，可以使用类型断言。
2. 与第三方库集成： 当你在 TypeScript 项目中使用第三方 JavaScript 库时，由于类型信息不完全，你可能需要使用类型断言来告诉 TypeScript 某些变量的类型。

类型断言有两种主要语法：

1. 尖括号语法（Angle Bracket Syntax）：

```
let value: any = "This is a string";
let length: number = (<string>value).length; // 将 value 断言为 string 类型
```

2. as 语法：

```
let value: any = "This is a string";
let length: number = (value as string).length; // 将 value 断言为 string 类型
```

注意事项和最佳实践：

- 尽量避免滥用类型断言。应该首先考虑使用类型推断和类型声明来提供更好的类型安全性。
- 如果你不确定类型断言是否正确，可以使用类型守卫或条件语句来检查变量类型。

```
if (typeof value === "string") {
    let length: number = value.length;
}
```

- 当你与第三方库集成时，可以使用类型声明文件（.d.ts 文件）来提供类型信息，而不必过度依赖类型断言。
- 使用 `as` 语法是更现代和被广泛接受的方式，推荐在 TypeScript 中使用 `as` 进行类型断言。

类型断言在处理复杂类型、第三方库集成或解决类型推断不准确的情况下非常有用，但应该小心使用，确保它们的正确性，并优先考虑更安全的类型推断和类型声明。

类型保护

在 TypeScript 中，类型保护（Type Guards）是一种机制，用于在运行时确定变量的类型，并在代码中以类型安全的方式处理不同类型的值。类型保护使你能够在处理联合类型、任意类型或未知类型的值时，更精确地了解它们的类型，从而避免潜在的运行时错误。

类型保护通常用于以下场景：

1. 联合类型的处理： 当一个变量的类型是联合类型（例如 `string | number`），你可以使用类型保护来确定它到底是哪种类型，并执行相应的操作。
2. 自定义类型检查： 你可以创建自定义的类型检查函数，这些函数根据特定的条件来判断变量的类型，并将其类型缩小为更具体的子类型。

以下是一些 TypeScript 中常见的类型保护方法：

1. typeof 类型保护： 使用 `typeof` 操作符可以用于检查基本数据类型，如 `string`、`number`、`boolean` 等。

```
function isString(value: any): value is string {
    return typeof value === "string";
}

let x: string | number = "hello";
if (isString(x)) {
    // 这里 x 被缩小为 string 类型console.log(x.toUpperCase());
}
```

2. instanceof 类型保护： 使用 `instanceof` 运算符可以用于检查对象是否属于某个类或构造函数的实例。

```
class Person {
    name: string;
    constructor(name: string) {
        this.name = name;
    }
}

function printName(obj: Person | string) {
    if (obj instanceof Person) {
        // obj 在这里被缩小为 Person 类型console.log(obj.name);
    } else {
        // obj 在这里被缩小为 string 类型console.log(obj.toUpperCase());
    }
}
```

3. 用户自定义类型保护函数： 你可以创建自己的函数来执行复杂的类型检查。

```
function isObject(value: any): value is { [key: string]: any } {
    return typeof value === "object" && value !== null;
}
```

```
4
5  function processValue(value: any) {
6      if (isObject(value)) {
7          // 在这里 value 被缩小为对象类型console.log(Object.keys(value));
8      } else {
9          console.log(value.toString());
10     }
11 }
```

类型保护允许你在运行时动态地确定变量的类型，以便更安全地处理不同类型的值，同时提高代码的可维护性和类型安全性。

声明合并

类型合并（Type Merging）是指在 TypeScript 中合并多个具有相同名称的接口（interface）或对象类型（object type）以创建一个新的类型，该类型包含了所有合并的部分。这允许你扩展或组合现有类型，以创建更复杂的类型。

类型合并通常发生在以下情况下：

1. 接口扩展：你可以使用 `extends` 关键字将一个接口扩展另一个接口，从而创建一个新的接口，它继承了两者的属性。

```
1  interface Car {
2    brand: string;
3    model: string;
4  }
5
6  interface Engine {
7    engineType: string;
8  }
9
10 interface CarWithEngine extends Car, Engine {
11   // Merged interface with properties from both Car and Engine
12 }
```

2. 对象字面量的合并：当你创建多个相同属性名的对象字面量时，它们将自动合并成一个对象类型，包含了所有属性。

```
1  const person = { name: "Alice" };
2  const contact = { email: "alice@example.com" };
3
4  const mergedPerson = { ...person, ...contact };
```

```
5 // The type of mergedPerson is { name: string, email: string }
```

3. 接口和类的合并：类可以同时实现接口，并且类的构造函数可以与接口合并，以便创建具有类实例方法和接口属性的类型。

```
1  class Person {
2    name: string;
3    constructor(name: string) {
4      this.name = name;
5    }
6  }
7
8  interface Greetable {
9    greet(): void;
10 }
11
12 interface Person extends Greetable {}
13
14 const alice = new Person("Alice");
15 alice.greet = function () {
16   console.log(`Hello, my name is ${this.name}`);
17 };
```

类型合并允许你创建更具表现力的类型，尤其在大型代码库中，这有助于组织和维护代码。当合并类型时，TypeScript会智能地合并相同属性名，确保类型安全。不过，请小心合并类型可能会引入复杂性，因此在使用时需要小心谨慎。

常用关键字

type

定义一个新的类型

```
1 interface A {
2   foo?: string;
3 }
4
5 type B = A;
```

typeof

获取值的类型

```
class A {}

let b: A = new A();
let c: typeof A = A;

interface B {
    foo: string;
    baz: boolean;
}

type m = B;
type n = typeof B;
```

keyof

`keyof` 操作符用于获取类型的所有属性名，结合索引访问类型可以从一个类型中获取属性的具体类型。

```
interface Person {
  name: string;
  age: number;
}

keyof typeof

type PersonKeys = keyof Person;  // "name" | "age"
type PersonNameType = Person['name'];  // string
```

在上面的示例中，我们使用 `keyof` 操作符获取了 `Person` 接口的属性名集合，并通过索引访问类型获取了 `Person` 接口中 `name` 属性的类型。

```
type Readonly<T> = { readonly [P in keyof T]: T[P]; }
```

extends

类型约束

在泛型中，我们可以使用 `extends` 关键字来对泛型类型进行约束。这样可以确保传递给泛型的类型满足特定条件。

```
function printProperty<T extends { name: string }>(obj: T): void {
  console.log(obj.name);
}

printProperty({ name: 'John', age: 25 });  // 输出 'John'
```

在上面的示例中，`printProperty` 函数接受一个泛型参数 `T`，该参数必须满足一个约束条件：具有 `name` 属性，且 `name` 的类型为 `string`。通过使用 `extends` 关键字和类型约束，我们可以确保 `obj` 参数具有所需的属性和类型，从而避免出现错误。

条件类型分支

除类型约束外, 还可以用在条件类型的分支判断中

```
T extends U ? X : Y
```

infer

`infer` 关键字用于在条件类型中推断类型，并将其赋值给一个类型变量。

```
type ReturnType<T> = T extends (...args: any[]) => infer R ? R : never;

function add(a: number, b: number): number {
  return a + b;
}

type AddReturnValue = ReturnType<typeof add>;  // 类型为 number
```

在上面的示例中，`ReturnType` 类型接受一个类型参数 `T`，并使用条件类型和 `infer` 关键字推断函数类型的返回类型。通过调用 `ReturnType<typeof add>`，我们推断出 `add` 函数的返回类型为 `number`。

never

`never` 类型是 TypeScript 中的一种特殊类型，通常用于表示绝对不会发生的情况。它有以下主要特征：

1. 绝对不返回值： 当一个函数永远不会正常返回时，可以将其返回类型标记为 `never` 。这通常在抛出不可恢复的错误或进入无限循环的情况下使用。

```
function throwError(message: string): never {
    throw new Error(message);
}

function infiniteLoop(): never {
    while (true) {
        // 无限循环
    }
}
```

2. 用于细化类型： 当 TypeScript 推断出某个分支将永远不会执行时，可以将变量标记为 `never` 类型，以细化类型检查。

```
function example(x: string | number) {
    if (typeof x === "string") {
        // 在这里，x 被细化为 string 类型
    } else if (typeof x === "number") {
        // 在这里，x 被细化为 number 类型
    } else {
        // 在这里，x 被细化为 never 类型
    }
}
```

3. 用于空数组或永不结束的循环： 当你创建一个永不结束的循环或一个永不返回元素的数组时，通常使用 `never` 类型。

```
function infiniteLoop(): never {
    while (true) {}
}

const emptyArray = [];
```

`never` 类型在类型系统中非常有用，因为它帮助捕获不合理的代码路径，同时也可以用于更严格的类型检查和提示。但要小心使用，确保只在确实表示不可能的情况下使用它。

readonly

`readonly` 是 TypeScript 中用于声明只读属性的关键字，它的作用是确保某个属性只能在对象创建时或构造函数中进行赋值，之后不可再被修改。它主要有以下作用：

1. 防止属性被修改： 使用 `readonly` 关键字可以防止对象的属性在对象创建后被修改。这在确保数据的不可变性上非常有用。

```
class Person {
    readonly name: string;

    constructor(name: string) {
        this.name = name; // 只能在构造函数中赋值
    }
}

const person = new Person("Alice");
person.name = "Bob"; // 编译错误，name 是只读属性
```

2. 类型检查和提示： 当你使用 `readonly` 关键字声明属性时，TypeScript 会提供更好的类型检查和代码提示，以确保你不会意外地修改只读属性。
3. 安全性： 在某些情况下，只读属性可以提高代码的安全性，防止在运行时意外地修改对象的状态。
4. 可用于接口和类： `readonly` 不仅可以应用于类的成员属性，还可以用于接口中的属性，以确保实现该接口的类不能修改这些属性。

```
interface Point {
    readonly x: number;
    readonly y: number;
}
```

-?

```
type Required<T> = {
    [P in keyof T]-?: T[P];
};
```

类型体操

实用类型体操举例

- *Awaited*
- *Partial*
- *Required*
- *Readonly*
- *Pick*
- *Record*
- *Exclude*
- *Extract*
- *Omit*
- *NonNullable*
- *Parameters*
- *ConstructorParameters*
- *ReturnType*
- *InstanceType*
- *Uppercase*
- *Lowercase*
- *Capitalize*
- *Uncapitalize*

类型体操库

- type-fest https://github.com/sindresorhus/type-fest

类型体操实践

总结

泛型和类型体操是 TypeScript 中强大的类型系统的关键组成部分。通过使用泛型，我们可以创建可重用、灵活和类型安全的代码。内置泛型函数提供了一些常用的类型转换工具，如 `Partial`、`Required` 和 `Pick`，可以帮助我们更方便地处理类型操作。

通过结合泛型、`extends` 关键字、内置泛型函数和其他高级类型概念，我们能够在 TypeScript 中编写更复杂、类型安全的代码，并利用 TypeScript 的强大类型系统来提高代码的可读性、可维护性和可扩展性。

Q&A

参考资料

- https://www.typescriptlang.org/docs/handbook/2/types-from-types.html
- https://cloud.tencent.com/developer/article/1916936
- https://developer.aliyun.com/article/1266032
- https://zhuanlan.zhihu.com/p/384172236
- https://www.icebreaker.top/articles/2021/9/26-ts-type-challenges-easy/
- https://frezc.github.io/2022/05/29/ts-type-system-programming/